MLFF PERF-P1 Shared Exact Selection Specification

Reusable FPS state, progressive coverage, and linear-memory DATA7 neighbor coverage

mdstats project

2026-08-15

Contents

1 Status

Gate: `PERF-P1`
Release: `mdstats 0.20.180a0`
Implementation status: complete for bounded supplied-data and synthetic wide/K-large authority
Next gate: `PERF-P2`

PERF-P1 consolidates exact deterministic selection and nested coverage state used by TARGET-DATA2C and DATA7. It changes execution realization only. Existing selector membership, tie rules, coverage mathematics, Stage-A decisions, DATA7 coverage content, and Class-E content digests remain scientific authority.

Authoritative implementation surfaces are:

- `mdstats.training_data.selection`;
- `mdstats.training_data.target_ladder`;
- `mdstats.training_data.target_coverage`;
- `mdstats.training_data.campaign_cli`;

- `benchmarks/benchmark_mlff_perf_p1.py`; and
- `tests/test_mlff_perf_p1.py`.

2 Authority boundary

PERF-P1 is Authority Class E. It introduces no new scientific schema and no new selection or coverage policy.

Area	Authority	Required invariant
Exact FPS workspace	E	Existing order, membership, and tie behavior
Quota-to-FPS handoff	E	Same quota prefix and continuation
Fused selector construction	E	Byte-identical FP64 matrix
Progressive coverage	E	Same family/rung statistics and report digests
DATA7 selected-neighbor state	E	Same nearest-other-selected distances and coverage
Coverage worker count	E	Execution-only; scientific digest invariant

Approximate nearest-neighbor search, relaxed tie tolerances, changed robust scales, altered quotas, changed selector normalization, reference subsampling, and changed scientific digests are outside this gate.

3 Exact FPS workspace

For selector matrix $X \in \mathbb{R}^{N \times D}$, PERF-P1 creates one `ExactFPSState` per deterministic selection run. Its persistent execution state contains:

```
row_norm_squared[N]
selected_mask[N]
selected_rank[N]
min_squared_distance[N]
selected_order[K_so_far]
lexical_uid_rank[N]
```

The row norms are computed once:

$$r_i = \|x_i\|_2^2.$$

When a new selected point x_j is appended, candidate squared distances use

$$d_{ij}^2 = r_i + r_j - 2x_i \cdot x_j.$$

The state then performs the historical deterministic maximin comparison with the existing FP64 tolerance and lexical UID tie resolution. Farthest-first traversal is a standard greedy construction for the metric k-center problem [1]; mdstats does not import a generic approximation policy from that theory. Its scientific authority is the pre-P1 project-specific deterministic order.

The legacy exact implementation remains test oracle. PERF-P1 qualification requires identical ordered prefixes, not merely equal selected sets.

4 Quota continuation and bounded centroid novelty

TARGET-DATA2C quota selection initializes the same `ExactFPSState` later used for FPS continuation. The quota prefix is therefore not reinserted into a new workspace.

Centroid novelty preserves the existing subtraction-plus-reduction numerical path, but processes bounded row blocks. This avoids materializing a full `X - centroid` temporary while preserving the historical floating-point operation semantics. A mathematically equivalent row-norm/matrix-vector formula was not made authoritative because a different BLAS/reduction path can perturb last-bit rounding and therefore deterministic ties.

5 Preallocated fused selector matrix

TARGET-DATA2C first determines the final fused selector width D, allocates one FP64 array of shape (N, D), and fills family slices directly. The former list-of-blocks plus `concatenate` realization remains an exact regression oracle.

For family blocks B_f of widths D_f,

$$D = \sum_f D_f,$$

and each block is copied into its assigned half-open column slice. Family order, normalization, missing channels, and all bytes of the resulting matrix remain unchanged.

No mmap-backed selector matrix is authorized by this release. That remains an optional execution extension if a later measured RAM budget requires it and exact equivalence is retained.

6 Progressive nested coverage

For one TARGET-DATA2B family, PERF-P1 scales the reference once and advances a state through nested selected prefixes. Let S_r be the selected set at rung r and let $A_r = S_r \setminus S_{r-1}$ be the newly added block. The maintained reference distance satisfies

$$d_r(x) = \min\left(d_{r-1}(x),\ d(x, A_r)\right).$$

Exact nearest-neighbor queries use `scipy.spatial.cKDTree.query` with `eps=0`; its `workers` argument is supplied from the campaign stage CPU budget rather than hard-coded to serial execution [2]. Worker count is execution-only and must not alter any report digest.

Selected minima/maxima and family state are likewise carried forward across rungs. Existing Wasserstein statistics continue to use SciPy's one-dimensional Wasserstein implementation [3]. PERF-P1 changes reuse, not the statistic.

The bounded supplied-data benchmark shows that this progressive realization is exact but is not faster for the four tested rungs on the current host. Median wall time rises from 0.643 s to 0.709 s, or 10.29%. The gate is retained because it removes repeated reconstruction, establishes one shared stateful path, and PERF-P1's measured performance acceptance is satisfied materially by exact FPS and DATA7 K-large memory/time reductions. No coverage-speed claim is made.

7 DATA7 linear persistent neighbor state

DATA7 needs, for every selected point, only the distance to its nearest *other* selected point. A dense matrix

$$D \in \mathbb{R}^{K \times K}$$

therefore stores information not needed by the coverage report and requires

$$8K^2 \text{ bytes}$$

in FP64. At $K = 8192$ this is exactly 512 MiB.

PERF-P1 persists only

`selected_neighbor_min_squared[K]`

requiring

$$8K \text{ bytes} = 64 \text{ KiB} \quad (K = 8192).$$

When a new block is appended, bounded pair blocks evaluate every old-new and new-new distance and update both endpoints' minima. No required pair is omitted. The persistent state is therefore $O(K)$ rather than $O(K^2)$ while the exact nearest-other-selected result is unchanged.

8 Qualification

8.1 Supplied-data scientific oracle

The complete PERF-P0 native reference is reused directly:

Quantity	Value
Target frames	37,633
Selector width	50
Coverage families	8
Tested nested rungs	128, 256, 512, 1024

Reference content digest:

`4f46dfaa6c366ede1cd4d19cf5fb8be65cfe49067b5aff85d668e0078915f9b8`

Exact results:

- fused selector matrix: byte-identical;
- exact FPS order through $K = 1024$: identical;
- all four coverage reports: identical content digests;
- coverage workers 1 and 4: identical report digests;
- regression TARGET-DATA2C plan/rungs: unchanged digests; and
- regression DATA7 selection/coverage: unchanged digests.

The PERF-P1 benchmark scientific digest is:

`ff08ca4aee884f1aaf4bf1969454bb75fc9e875eb8c29d57c46fe0100dadb12e`.

8.2 Matched CPU measurements

The host exposes an Intel Xeon Platinum 8573C, nine visible logical CPUs, an 8-core cgroup quota, and a 4 GiB memory limit. Full-reference stages use five matched repetitions.

Path	Legacy median wall	PERF-P1 median wall	Change
Fused selector assembly	0.104 s	0.105 s	+0.98%
Exact FPS, $K = 1024$	1.550 s	0.657 s	**-57.61%**
Four-rung coverage	0.643 s	0.709 s	+10.29%

The assembly timing is effectively neutral on this host, while median measured RSS increment falls from 30.25 MiB to 0.008 MiB after allocator warm-up because the final concatenation temporary is removed. RSS increment is allocator- and page-state-sensitive, so it is evidence of removed transient allocation rather than a portable memory bound.

8.3 Wide exact FPS

For a deterministic synthetic selector matrix of shape 4000×128 and $K = 512$:

Path	Wall time
Legacy exact FPS	0.1821 s
PERF-P1 exact FPS	0.0487 s

The exact order digest is unchanged and wall time improves by **73.25%**.

8.4 DATA7 K-large case

For deterministic $K = 8192$, $D = 16$ selected-neighbor coverage:

Metric	Dense legacy	PERF-P1
Persistent state	512 MiB	64 KiB
Wall time	1.954 s	0.506 s
Peak RSS	1149.89 MiB	637.22 MiB

Persistent state is **8192x smaller**, wall time improves by **74.09%**, and peak RSS is **44.58% lower**. Final nearest-neighbor minima are byte-identical.

9 Acceptance decision

PERF-P1 passes bounded qualification because:

1. quota/FPS prefixes and exact FPS orders are unchanged;
2. selector matrices and coverage reports are unchanged;
3. deterministic tie authority is unchanged;
4. Stage-A-relevant coverage evidence is unchanged;
5. DATA7 selected-neighbor results are exact;
6. worker count is execution-only; and
7. realistic wide/K-large cases show material wall-time and memory reductions.

The measured four-rung progressive-coverage slowdown is recorded as a non-blocking performance limitation. PERF-P2 must not assume that progressive state alone makes coverage faster.

10 Evidence and limitations

Evidence is stored in:

- `audits/analysis/mlff_perf_p1_lta_cloud_cpu_2026-08-15.{json,md,pdf}`;
- `release/MLFF_PERF_P1_QUALIFICATION_0.20.180a0.json`; and
- this specification.

No authorizing MACE-MH-1 checkpoint or GPU runtime was supplied. PERF-P1 makes no TRAIN2/EVAL2, GPU-memory, GPU-throughput, OOM, or full production-campaign performance claim.

11 References

[1] T. F. Gonzalez, "Clustering to Minimize the Maximum Intercluster Distance," *Theoretical Computer Science* **38**, 293–306 (1985). DOI: 10.1016/0304-3975(85)90224-5.

[2] SciPy developers, "scipy.spatial.cKDTree.query," SciPy reference documentation. Available at: https://docs.scipy.org/doc/scipy/reference/generated/scipy.spatial.cKDTree.query.html (accessed 2026-08-15).

[3] SciPy developers, "scipy.stats.wasserstein_distance," SciPy reference documentation. Available at: https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.wasserstein_distance.html (accessed 2026-08-15).